计算机系统基础 PA1 实验报告

实验人：葛家韬（241220071）
授课教师：汪亮 苏丰

摘要

本实验围绕《计算机系统基础》课程的PA1阶段任务展开，涵盖了数据表示与存取、整数运算模拟以及浮点运算模拟三个部分。在PA1-1中，通过阅读与实现CPU_STATE结构体，理解了struct与union在寄存器模拟中的作用，特别是union能更好地模拟x86架构中不同位宽寄存器共享存储空间的特性，同时减少内存开销；在PA1-2中，完成了算术逻辑单元（ALU）的加减乘除、带进/借位、逻辑和移位等运算，并维护各类标志位，通过对溢出与进位等边界情况的处理，加深了对整数机器级运算机制的理解；在PA1-3中，基于IEEE 754标准实现了浮点数运算单元（FPU）的模拟，处理了规格化数、非规格化数、零、无穷和NaN等特殊情况，并实现加减乘除与比较等功能，尤其在规格化与舍入处理上需要谨慎设计与验证，从而更加深入理解了浮点数表示及其运算的复杂性。整体而言，本实验不仅强化了对数据表示、整数与浮点运算等核心知识的掌握，也提升了对底层抽象设计、边界情况处理及逻辑正确性验证的能力，为后续实验奠定了坚实基础。

PA 1-1 数据的表示和存取

实验问题

结构体的参考实现为什么把部分struct 改成了union?

将struct改为union，可以更好的模拟x86架构寄存器的模式。x86的一些通用寄存器不是相互独立的存储单元，而是共享一块存储空间的不同视角，如低8位是AL，高8位是AH，合并起来就是AX，完整的32位就是EAX。在模拟这种结构的过程中，使用union可以让这些不同位宽，不同偏移量的寄存器指向同一块存储区域（模拟程序的同一段内存空间），从而保证了无论访问的是8位、16位还是32位的寄存器的时候，访问和修改的都是同一块区域的数据。同时，union还避免了不必要的内存开销，所有成员共用内存，而不是像struct一样各自占用空间，可以压缩存储空间并降低内存复制的开销，如在对eax、ecx赋值的过程中，相当于对gpr[i]进行了同步的操作，而无需再通过代码实现这种同步。

课程知识点

- 数据的类型及其机器级表示
- 机器数的存取：内存的模拟
- 机器数的存取：寄存器的模拟

心得体会

进行了对CPU_STATE结构体的阅读与理解，我对计算机底层的数据组织方式有了更深刻的体会。结构体与联合体的嵌套不仅展示了寄存器在不同位宽下的访问方式，让我意识到抽象的数据结构如何直观映射到底层硬件特性。在分析的过程中，理解了同一块存储区域可以被不同视角解读，这种设计既体现了灵活性，也突出了程序员在抽象硬件时的构思。

PA1-2 整数的表示和运算

课程知识点

- 无符号整数和带符号整数的机器级表示
- NEMU对ALU的模拟
- 以ADD运算为例，在模拟ALU中实现运算功能并测试

实验流程

首先设置各类标志位，ZF判断结果是否为零，SF取符号位，PF判断结果低8位是否含偶数个1；然后实现加法alu add，设置CF与OF，其中CF由进位决定，OF由同号相加溢出确定；在alu adc中需要额外考虑CF的进位；减法alu sub和带借位的alu sbb与加法类似，只是方向相反，sbb的CF相当于逆时针循环；乘法alu mul使用无符号64位运算，高位若非零则CF与OF都置1，而alu imul使用有符号64位乘法即可；除法alu div与alu idiv中若除数为0返回0，结果按数据宽度截断后返回无符号或有符号值；各种位操作逻辑简单；移位操作中，alu shl与alu sal为左移，CF由最高位移出值决定，而alu shr与alu sar为右移，其中sar需进行符号扩展。

心得体会

这部分PA真的超级繁琐，需要考虑的问题特别多，每一个FLAG，每一个符号。这一部分在实现多个函数时，需要充分考虑溢出等特殊情况，才能正确完成不同标志位的更新。通过这一过程，我对运算中可能出现的临界与特殊情况有了更加深入的理解。

PA1-3

实验问题

对应输入是规格化或非规格化数，而输出产生了阶码上溢结果为正（负）无穷的情况

- 加法：最大的规格化数0x7F7FFFFF + 0x7F7FFFFF结果大于浮点数能表达的最大正数，会发生溢出，成为正无穷
- 乘法：最大的规格化数0x7F7FFFFF * 0x7F7FFFFF结果大于浮点数能表达的最大正数，会发生溢出，成为正无穷

对应输入是规格化或非规格化数，而输出产生了阶码下溢结果为正（负）零的情况。

- 加法：不存在两个数相加发生下溢，不论非规格化数与规格化数如何相加，结果不会急剧下降变成一个绝对值小于最小非规格化数的数
- 乘法：最小非规格化数0x00000001 * 0x00000001结果小于浮点数可以表示的最小正数，会发生溢出，成为正0

课程知识点

- IEEE 754 浮点数标准
- 浮点数的运算模拟

实验流程

实现了浮点数运算单元（FPU）模拟器，主要功能包括：通过uint32_t与float的互转实现浮点数的底层表示操作；定义并处理正零、负零、正无穷、负无穷、NaN等特殊值；提供浮点数的加、减、乘、除的内部实现，包含规格化、舍入、溢出、非规格化数等情况的处理，以及特殊情形（如零与无穷相乘）的判定；维护FPU的寄存器栈，支持基本的压栈、出栈、交换、复制等操作；提供对浮点数的算术运算（add、sub、mul、div）和比较（cmp、cmpi）的封装函数，并更新相应的FPU状态标志或CPU标志位。整体上，该代码在底层位运算和状态管理的基础上，模拟了硬件FPU对浮点数运算的行为。

心得体会

在实现这一部分时遇到了一些困难，例如在规格化函数的设计中，不仅需要充分理解规格化的基本原理，还要结合浮点数的存储格式与运算特点进行深入分析。实现过程中必须格外谨慎，稍有疏忽就可能导致结果出现偏差，特别是涉及GRS位的功能与实现，它们直接影响舍入的准确性和最终的计算结果。因此在编写代码的同时，还需要不断进行测试和验证，以确保逻辑的正确性和稳定性。完成这一环节后，不仅对浮点数的运算机制，尤其是规格化处理的细节有了更深入的体会，同时也进一步理解了浮点数标准设计背后的思考与精妙之处，这为后续的运算模块实现提供了宝贵的经验和思路。

参考资料及声明

- 课程群中ppt资料PA1-1 PA1-2 PA1-3
- PA框架代码
- 张逸凯（171840708）实验报告 https://gitee.com/whutglc/NJUCS-Course-Material-from-YikaiZhang
- 陈星宇（241220058）实验报告
- 实验过程中使用了ChatGPT提供思路与指导，实验报告中的部分排版与表达参考了ChatGPT生成的内容。所有核心实验过程与结论均由本人亲自完成并负责。